El trie

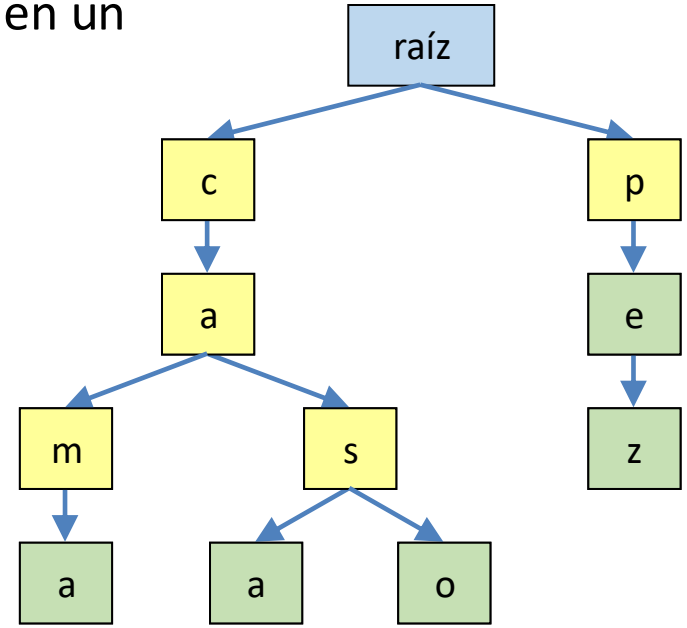
Un árbol de letras para claves de texto

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

El trie

El árbol compara cadenas **enteras**; la tabla hash convierte la cadena **entera** en un número. Un **trie** (re**trie**val, árbol de prefijos) la recorre **letra a letra**.



Es el **árbol general** del tema 4, en el que:

- cada arista lleva una **letra**
- el camino desde la raíz hasta un nodo deletrea un **prefijo**
- cada nodo tiene una marca **finDePalabra**: "aquí termina una clave"

Lo que aporta:

- **buscar es O(L)**, con L la longitud de la clave, sea cual sea n
- los **prefijos comunes** se guardan una vez
- **todas las claves que empiezan por…**: lo que una tabla hash de strings no sabe hacer

¿Por qué el color verde de los nodos?

¿Para qué nos puede servir un Trie?

06_trie.cpp

3 alternativas

1. TrieArray: Trie con Nodo* hijos[26]: Un array de 26 Nodos, uno por posible letra.
   - Acceso directo O(1) por letra, pero mucho espacio desperdiciado.
2. Trie<map<char, Nodo*>> hijos: Un árbol rojo-negro por nodo.
   - Acceso O(log n) por letra y todo con orden.
3. Trie<unordered_map<char, Nodo*>> hijos: Un hash+encadenamiento por nodo.
   - Acceso O(1) por letra pero sin orden.

El 2. y el 3. se parecen, se pueden agrupar en uno y esperar el tipo de map con template:

```
template <template <class...> class Mapa> // Mapa no es un tipo, es una plantilla a la que
class Trie {                              // todavía le faltan sus parámetros.
```

Desde el main:

```
Trie<std::map> opcion2;
Trie<std::unordered_map> opcion3;
```

Cuentas para máquina de 64 bits con libc++

- Nodo vacío

Nodo	Contenido	Suma	Con relleno
TrieArray	Nodo* hijos[26] → 26 × 8 = 208, más bool = 1	209	216 (se redondea al múltiplo de 8)
Trie<map>	std::map = 24, más bool = 1	25	32
Trie<unordered_map>	std::unordered_map = 40, más bool = 1	41	48

  - **std::map (24 bytes):** un puntero al primer nodo (begin), un nodo centinela que es solo un puntero a la raíz, y un size_t con el tamaño. 3 × 8 = 24.
  - **std::unordered_map (40 bytes):** un puntero al array de cubetas (8) y el contador de cubetas (tipo size_t: 8 bytes), un puntero a la cabeza de la lista de nodos (8), un size_t con el tamaño (8), y max_load_factor como float (4 + 4 de relleno). Total 40.

- Lo que cuesta cada hijo, además del Nodo vacío

std::map: un nodo de árbol rojinegro de 48 bytes

- punteros left, right y parent: 24; bool is_black: 1, que con relleno ocupa 8 → 32
- pair<const char, Nodo*>: 1 + 7 de relleno + 8 = 16;total: **48 bytes**

std::unordered_map: un nodo de hash de 32 bytes, más las cubetas

- puntero next: 8; hash guardado (size_t): 8;pair<const char, Nodo*>: 16; total: **32 bytes**
- aparte está el array de cubetas, de 8 bytes cada una.
- El número de cubetas crece por primos

hijos	1–2	3–5	6–11	12–23	24–26
cubetas	2	5	11	23	47
bytes	16	40	88	184	376

Cuentas para máquina de 64 bits con libc++

Total de la estructura de hijos de un nodo con *k* hijos

Aquí no cuento los Nodo hijos, que se pagan igual en las tres versiones.

- **Array:** 216, fijo, sea cual sea *k*
- **map:** 32 + 48·k
- **unordered_map:** 48 + 32·k + 8·cubetas(k)

k	Array	map	unordered_map
0 (hoja)	216	32	48
1	216	80	96
2	216	128	128
3	216	176	184
4	216	224	216
5	216	272	248
26	216	1280	1256

Conclusión: con 3 hijos o menos, el map gasta menos que el array. El unordered_map empata con el array en 4 hijos. En un trie de palabras casi todos los nodos tienen 0 o 1 hijos, así que las versiones con diccionario ahorran mucha memoria.

Todo el trie: Cada nodo, salvo la raíz, es hijo de exactamente un padre, así que el árbol tiene $N - 1$ aristas en total, repartidas como sea:

- **Array:** 216·N
- **map:** 32·N + 48·(N − 1) ≈ **80 bytes por nodo**
- **unordered_map:** 48·N + 32·(N − 1) + todas las cubetas ≈ **80 bytes por nodo + 8 bytes * total cubetas**

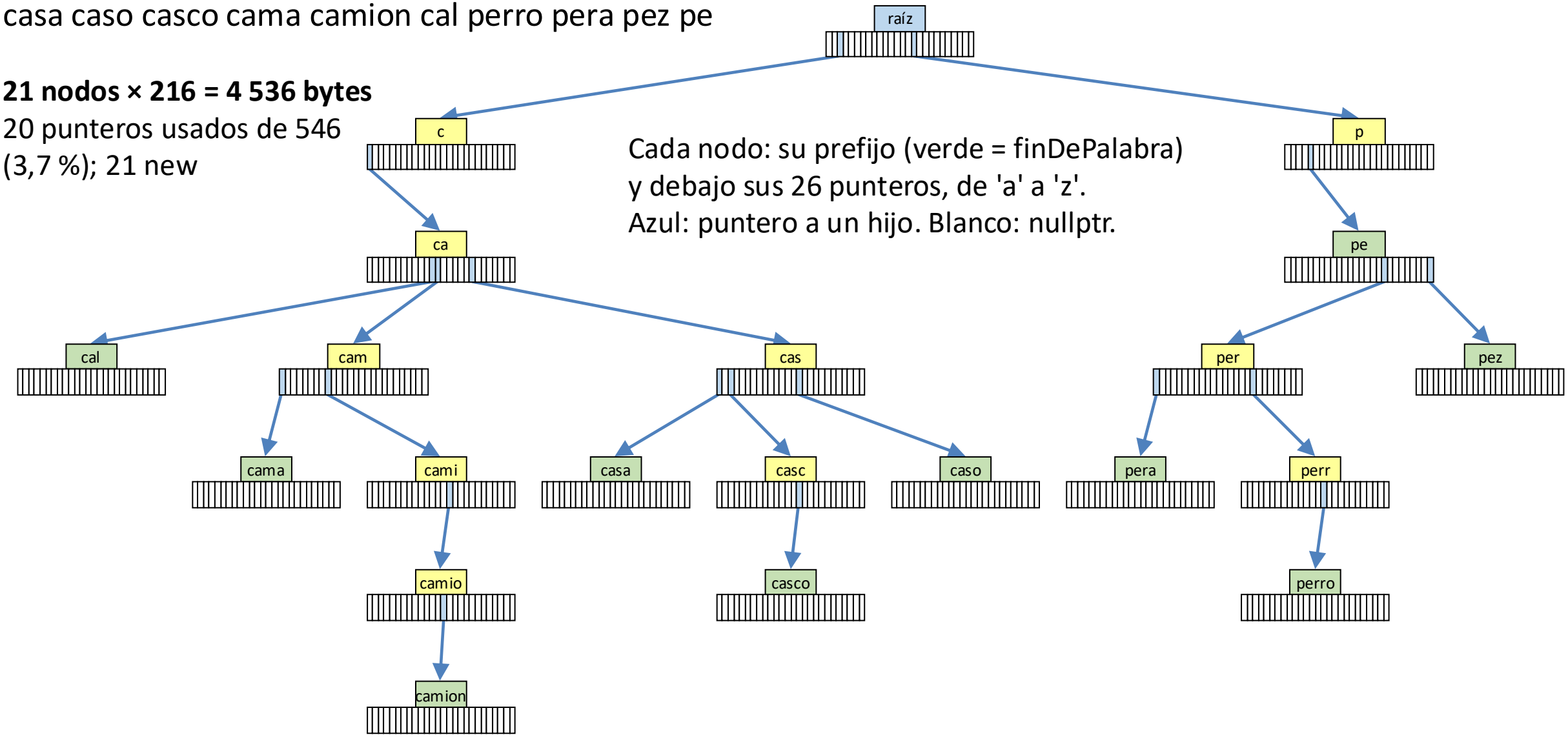
Aunque cada nodo del map cueste distinto, la memoria total depende solo de *N* y no de la forma del árbol. Un nodo con muchos hijos es más caro, pero eso se compensa porque sus hijos suelen ser hojas baratas. De media, el map gasta unas 2,7 veces menos que el array (216 / 80).

Trie con Nodo* hijos[26]

casa caso casco cama camion cal perro pera pez pe

21 nodos × 216 = 4 536 bytes
20 punteros usados de 546
(3,7 %); 21 new

Cada nodo: su prefijo (verde = finDePalabra)
y debajo sus 26 punteros, de 'a' a 'z'.
Azul: puntero a un hijo. Blanco: nullptr.



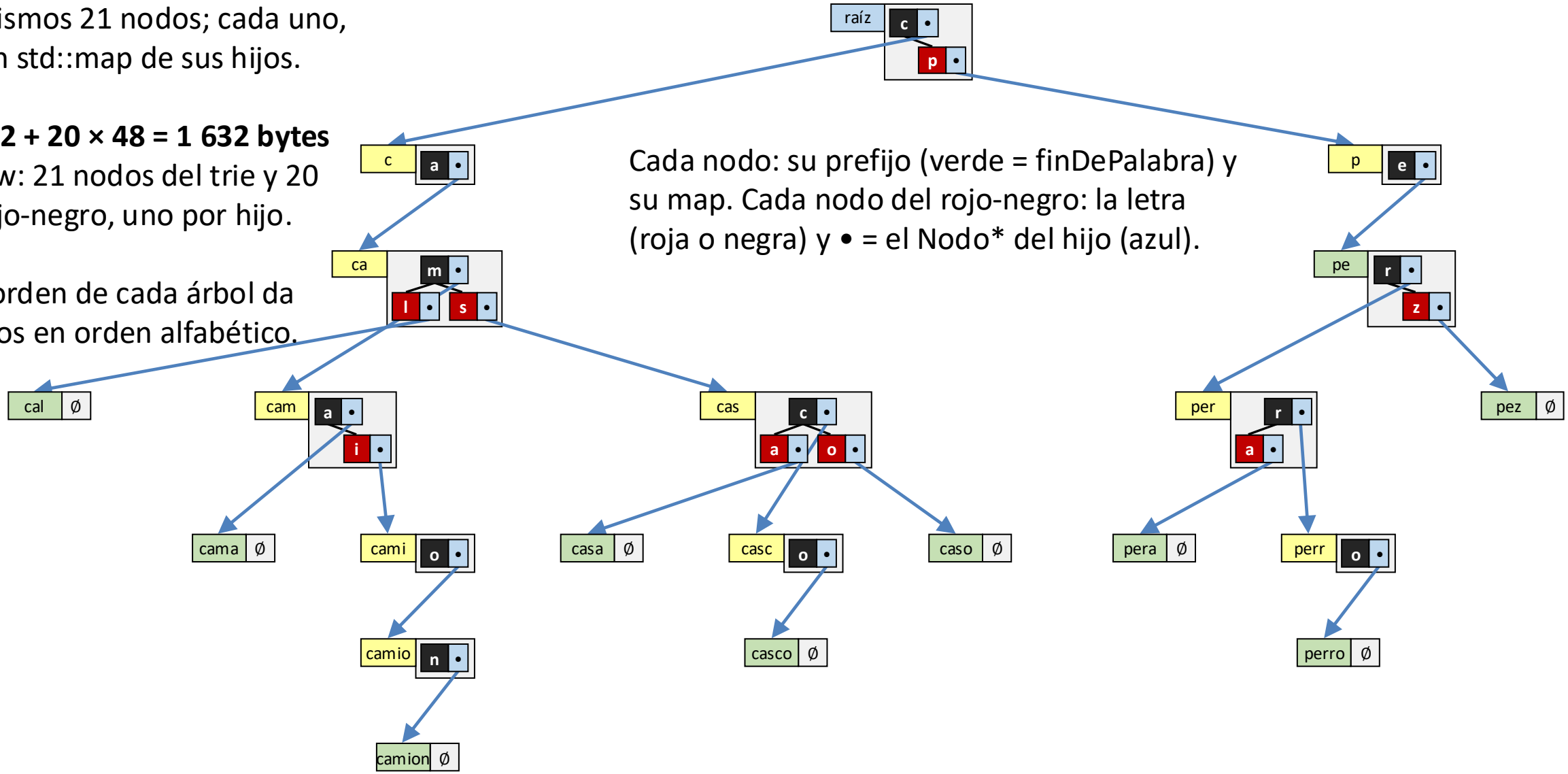
Trie<map>: un rojo-negro por nodo

Los mismos 21 nodos; cada uno, con un std::map de sus hijos.

21 × 32 + 20 × 48 = 1 632 bytes
41 new: 21 nodos del trie y 20 del rojo-negro, uno por hijo.

Un inorden de cada árbol da los hijos en orden alfabético.

Cada nodo: su prefijo (verde = finDePalabra) y su map. Cada nodo del rojo-negro: la letra (roja o negra) y • = el Nodo* del hijo (azul).



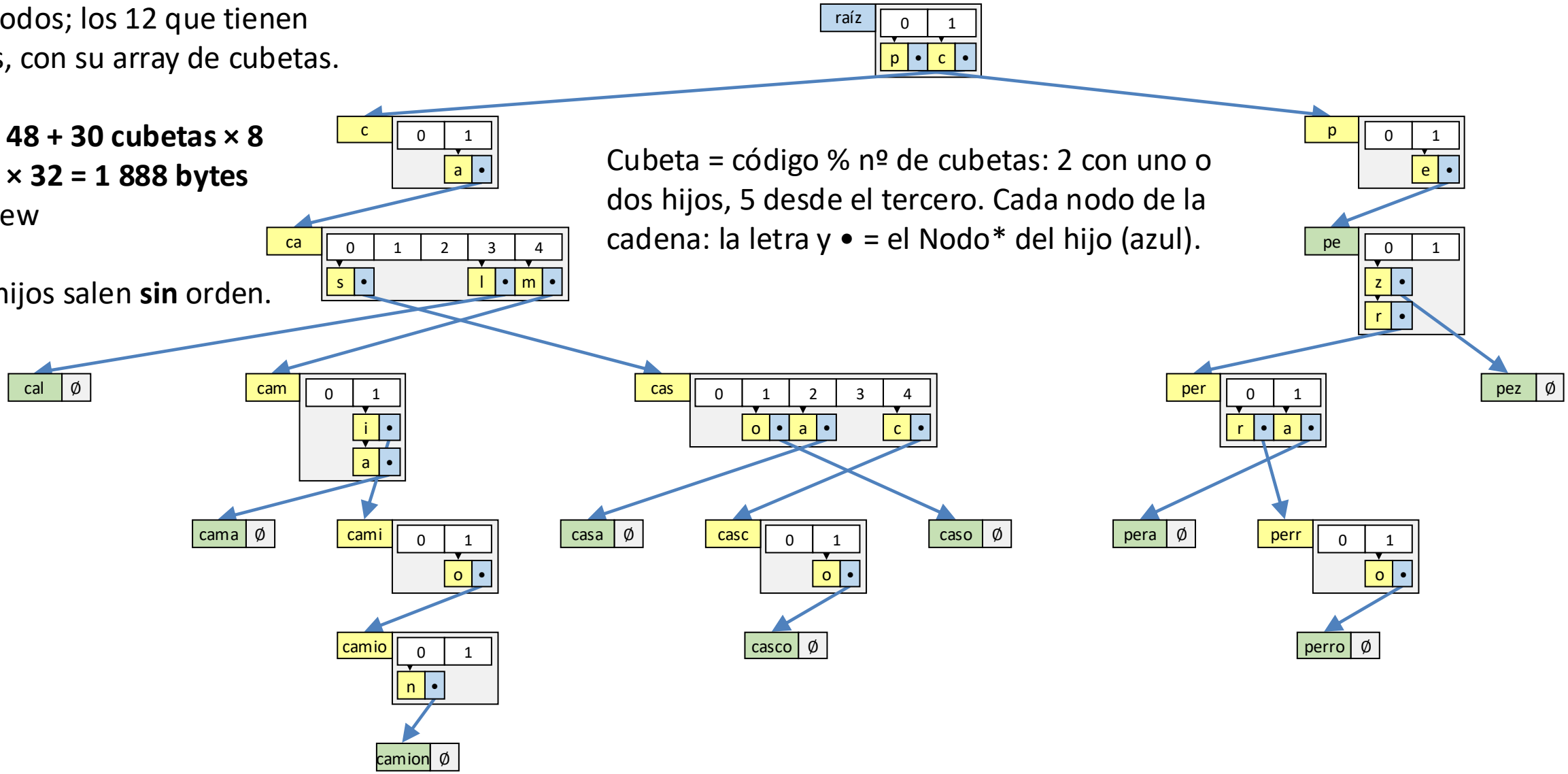
Trie<unordered_map>: tabla hash por nodo

21 nodos; los 12 que tienen hijos, con su array de cubetas.

21 × 48 + 30 cubetas × 8 + 20 × 32 = 1 888 bytes
53 new

Los hijos salen **sin** orden.

Cubeta = código % nº de cubetas: 2 con uno o dos hijos, 5 desde el tercero. Cada nodo de la cadena: la letra y • = el Nodo* del hijo (azul).



Tres tries, las mismas cuatro palabras

Con 11 nodos de trie (raíz incluida) y **10 hijos** en total.

Hijos en…	Nodo	Por hijo	Cubetas	new	Bytes
Nodo* hijos[26]	216	—	—	11	2 376
map<char, Nodo*>	32	48	—	21	**832**
unordered_map<char, Nodo*>	48	32	7 × 16	28	960

El array paga por las letras que **podrían** estar; los mapas, por las que **están**.
A cambio, cada hijo es un **new** más, y bajar una letra es un salto más por memoria.

06_trie.cpp

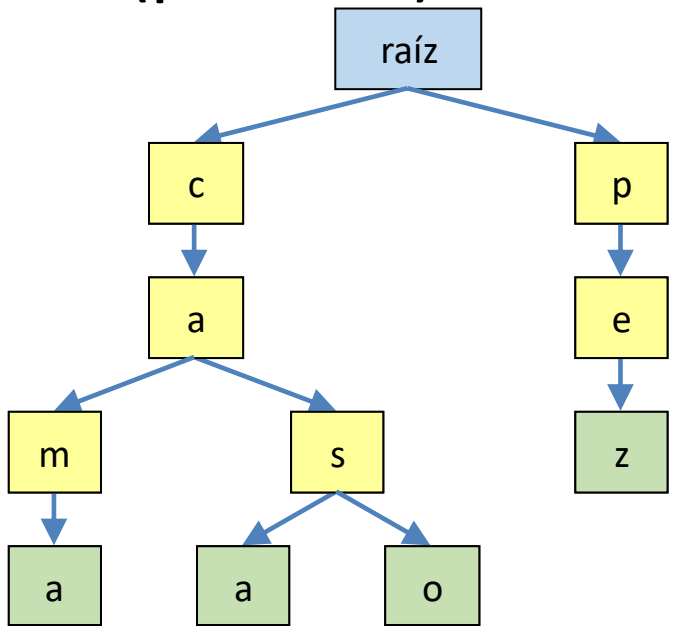
Implementación

Trie

```
void insertar(const string& p) {
    Nodo* n = raiz_;
    for (char c : p) {
        Nodo*& h = n->hijos[c];
        if (h == nullptr)
            h = new Nodo;
        n = h;
    }
    n->finDePalabra = true;
}
```

4 palabras, 15 letras, 11 nodos (incluyendo la raíz): "ca" y "cas" sirven a varias palabras.

insertar(palabra)



verde = finDePalabra (ahí termina una clave)

el c [illegible] "casa"

Implementación

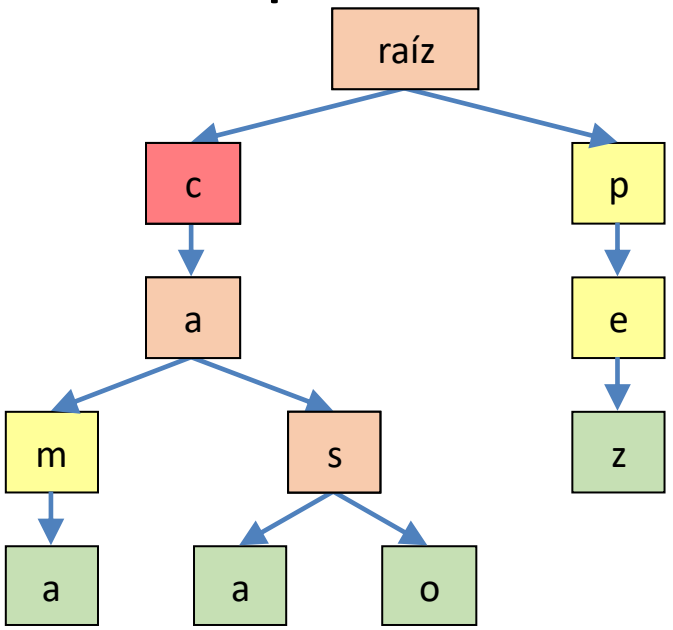
Trie

```
bool buscar(const string& p) {
    const Nodo* n = bajar(p);
    return n && n->finDePalabra;
}
bool empiezaPor(const string& p) {
    return bajar(p) != nullptr;
}
```

Bajar cuesta O(L): una letra por nivel, sin mirar las demás claves.

Ser PREFIJO no es ser CLAVE: para eso está finDePalabra.

buscar / empiezaPor



verde = finDePalabra (ahí termina una clave)

buscar(cap) → baja c → a → p → true (verde)

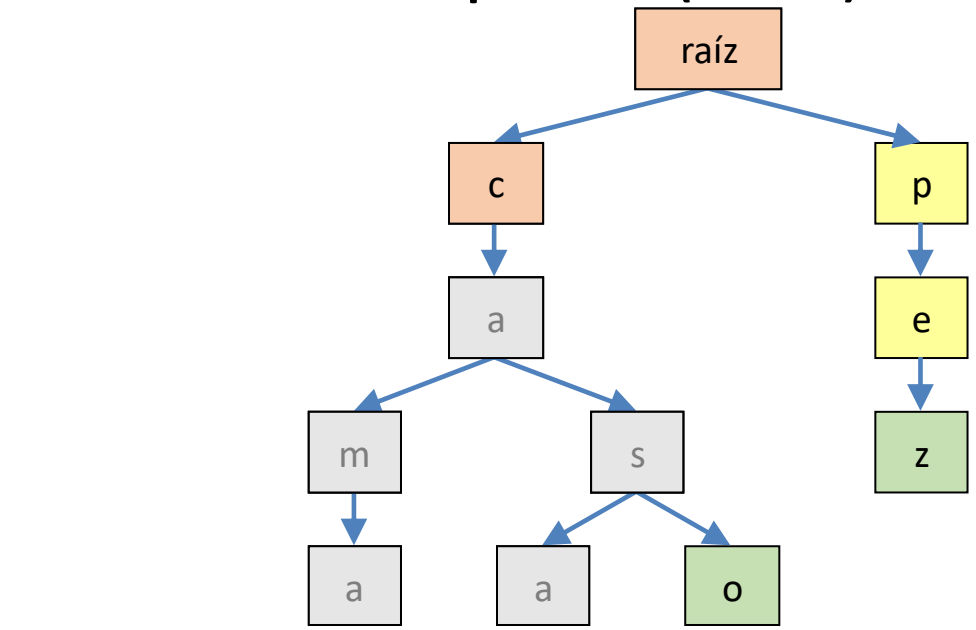
Implementación

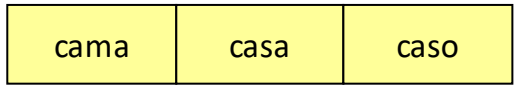
Trie

```
void recoger(const Nodo* n,
             string& palabra,
             vector<string>& salida) {
    if (n->finDePalabra)
        salida.push_back(palabra);
    for (auto& [letra, hijo] : n->hijos) {
        palabra.push_back(letra);
        recoger(hijo, palabra, salida);
        palabra.pop_back();
    }
}
```

Un PREORDEN del subárbol del prefijo.
Con los hijos en un std::map, las palabras salen ORDENADAS, gratis.

autocompletar("ca")





bajar("ca", "ca", salida) → subárbol

¿Cómo guarda cada nodo a sus hijos?

La pregunta del árbol general del tema 4, con tres respuestas en 06_trie.cpp:

Hijos en…	**Bajar una letra**	**Memoria por nodo**	**Orden**
Nodo* hijos[26]	O(1)	216 bytes, siempre	alfabético
std::map<char, Nodo*>	O(log σ)	32 + un nodo por hijo	**alfabético**
std::unordered_map<char, Nodo*>	O(1) esperado	48 + cubetas + nodos	**ninguno**

Las dos últimas son **la misma clase**: el tipo de diccionario es un parámetro.

```
template <template <class...> class Mapa>
class Trie { struct Nodo { Mapa<char, Nodo*> hijos; bool finDePalabra; }; … };

Trie<std::map> b;      Trie<std::unordered_map> c;
```

06_trie.cpp

Ejercicio 5: autocompletar

Completa recoger() en 06_trie.cpp.

¿Qué recorrido es? ¿Qué se le hace a "palabra" al bajar y qué al volver?
¿Salen en el mismo orden con Trie<map> que con Trie<unordered_map>?

Un **preorden**: visitar (si finDePalabra, a la salida) y luego cada hijo.
palabra.push_back(letra) al bajar, pop_back() al volver: la cadena es una **pila**.

Trie<map>, "ca": cal cama camion casa casco caso
Trie<unordered_map>, "ca": cal camion cama casco caso casa

06_trie.cpp

Frente a frente

Árbol, tabla hash y trie, con las mismas claves

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

100 000 palabras, tres familias

	construir	memoria	bytes/pal.	buscar	prefijos
ÁRBOL std::map	24,7 ms	6,1 MB	64	42,2 ms	7,69 ms
HASH unordered_map	**6,8 ms**	**5,4 MB**	56	**5,9 ms**	164 ms
TRIE hijos[26]	25,1 ms	64,1 MB	673	9,9 ms	**0,00 ms**
TRIE hijos en map	42,9 ms	23,8 MB	249	36,1 ms	**0,01 ms**
TRIE hijos en unordered	51,1 ms	28,0 MB	293	35,7 ms	**0,01 ms**

buscar: 100 000 que están y 100 000 que no. prefijos: 300 consultas "¿cuántas empiezan por…?"
Memoria: bytes pedidos con new (se sustituye el operator new global).

```
0_palabras_todas.txt: 646616 lineas, 14587 descartadas (caracteres fuera de a-z), 537511 palabras distintas
=== 268755 palabras (media 10.2 letras), 268755 busquedas con exito, 268755 sin exito,
    300 consultas por prefijo ===

                              construir    memoria     bytes/     buscar     prefijos
                                   (ms)       (MB)    palabra       (ms)         (ms)
  ARBOL   std::map                73.8       16.4       64.0      152.8        22.27
  HASH    std::unordered_map      22.5       15.4       60.3       16.2       567.04
  TRIE    hijos[26]               61.7      123.6      482.3       35.0         0.00
  TRIE    hijos en map           141.7       45.8      178.6      158.6         0.02
  TRIE    hijos en unordered     176.5       53.3      208.1      177.0         0.01
```

07_comparativa_diccionarios.cpp

Cómo leer la comparativa

Buscar: gana la tabla hash. El trie de 26 punteros va cerca (un acceso a array por letra). Con map o unordered_map, el trie paga **un contenedor por letra** y empata con el árbol.

Prefijos: la tabla hash pierde por goleada. **No tiene orden**: mira las n claves en cada consulta. El árbol recorre un tramo contiguo desde lower_bound(p): O(log n + k).
El trie baja |p| letras y lee un contador: O(|p|).

Memoria: el trie **no guarda las cadenas** y aun así es el que más gasta.
Compartir prefijos ahorra letras (3,1 nodos por palabra en vez de 9,7), pero un nodo cuesta mucho más que una letra.

07_comparativa_diccionarios.cpp

La tabla final del tema

TAD	En la STL	Por debajo	Buscar
Diccionario ordenado	map, set	Árbol rojo-negro	O(log n) garantizado
Diccionario sin orden	unordered_map, _set	Hash + encadenamiento	O(1) esperado, O(n) peor
Cadenas por prefijo	(no está)	Trie	O(longitud)

Cómo elegir, en orden:

1. ¿Solo clave exacta? → **unordered_map** (con reserve si sé cuántas)
2. ¿Orden, siguiente/anterior, rangos? → **map**
3. ¿Cadenas y muchas consultas por prefijo? → un **trie**
   (si son pocas, un map con lower_bound basta y gasta mucho menos)
4. ¿Claves enteras 0..n−1? → un **vector**: direccionamiento directo

Y fuera de clase

La STL usa encadenamiento por compatibilidad, no por velocidad: el estándar promete que las referencias sobreviven a un rehash, y eso exige un nodo por elemento.
Las tablas más rápidas son de direccionamiento abierto: absl::flat_hash_map (Swiss tables), boost::unordered_flat_map, Robin Hood.

Ataques de colisión: quien conoce la función hash puede fabricar claves que caigan todas en la misma cubeta (el HashPésimo, a propósito). Python y Rust usan una semilla aleatoria por ejecución; el HashMap de Java convierte en árbol las cubetas de más de 8.

Tries de verdad: radix trees (tries compactos), en routers para el prefijo más largo de las tablas de rutas IP, en el núcleo de Linux y en bases de datos en memoria.

Resumen

El **direccionamiento directo** da O(1) exacto, con memoria del universo de claves.

Una **función hash** lleva cualquier clave a una de m casillas: mod m con m primo, polinómica (Horner) para cadenas. Las **colisiones** son inevitables, y llegan pronto.

Encadenamiento: $O(1 + \alpha)$, y **rehash** al doble: O(1) amortizado (tema 2).

Direccionamiento abierto: mejor caché, $\alpha < 1$, agrupamiento y **lápidas**.

unordered_map: O(1) **esperado** y **sin orden**.

El **trie**: O(longitud) y prefijos baratos, a cambio de memoria.